

[Demos](#) / [Getting Started](#) / [Compilation of quantum circuits](#)**Have your say!**

Enjoying PennyLane? Take the
[2026 Unitary Foundation Quantum
Open Source Software Survey](#) now.

Compilation of quantum circuits



Borja Requena

Published: June 13, 2023. Last updated: April 17, 2026.

Quantum circuits take many different forms from the moment we design them to the point where they are ready to run on a quantum device. These changes happen during the compilation process of the quantum circuits, which relies on the fact that there are many equivalent representations of quantum circuits that use different sets of gates but provide the same output.

Out of those representations, we are typically interested in finding the most suitable one for our purposes. For example, we usually look for the one that will incur the least amount of errors in the specific hardware on which we are compiling the circuit. This usually implies decomposing the quantum gates in terms of the native ones of the quantum device, adapting the operations to the hardware's topology, combining them to reduce the circuit depth, etc.

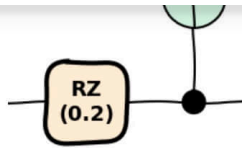
A crucial part of the compilation process consists of repeatedly performing minor circuit modifications. In PennyLane, we can apply `transforms` to our quantum functions in order to obtain equivalent ones that may be more convenient for our task. In this tutorial, we introduce the most fundamental transforms involved in the compilation of quantum circuits.

Circuit transforms ¶

When we implement quantum algorithms, it is typically in our best interest that the resulting circuits are as shallow as possible, especially with the currently available noisy quantum devices. However, they are often more complex than needed, containing multiple operations that could be combined to reduce the circuit complexity, although it is not always obvious. Here, we introduce three simple `transforms` that can be implemented together to obtain simpler quantum circuits. The transforms are based on very basic circuit equivalences:



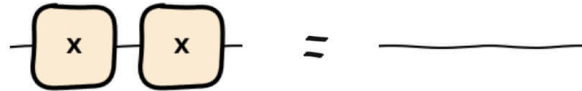
Permute commuting operators



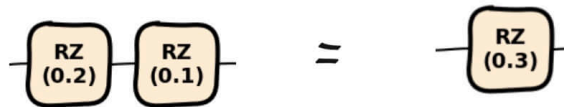
Have your say!

Enjoying PennyLane? Take the [2026 Unitary Foundation Quantum Open Source Software Survey](#) now.

Cancel consecutive inverse operators



Combine consecutive operators



To illustrate their combined effect, let us consider the following circuit.

```
import pennylane as qn
import matplotlib.pyplot as plt

dev = qn.device("default.qubit", wires=3)

@qn.qnode(dev)
def circuit(angles):
    qn.Hadamard(wires=2)
    qn.Hadamard(wires=1)
    qn.CNOT(wires=[1, 0])
    qn.CNOT(wires=[2, 1])
    qn.RZ(angles[1], wires=2)
    qn.RY(angles[0], wires=0)
    qn.CNOT(wires=[2, 1])
    qn.RY(-angles[0], wires=0)
    qn.RX(angles[0], wires=0)
    qn.RZ(-angles[1], wires=2)
    qn.CNOT(wires=[1, 0])
    qn.RX(angles[0], 0)
    qn.Hadamard(wires=1)
    qn.CY(wires=[1, 2])
    qn.CNOT(wires=[1, 0])
    return qn.expval(qn.PauliZ(wires=0))
```

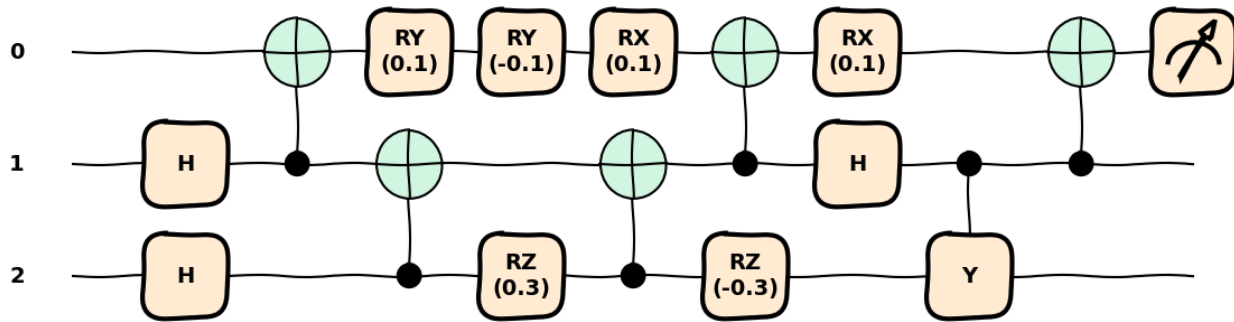


```
angles = [0.1, 0.3, 0.5]
```

```
qp.draw_mpl(circuit, decimals=1, style="sketch")
plt.show()
```

Have your say!

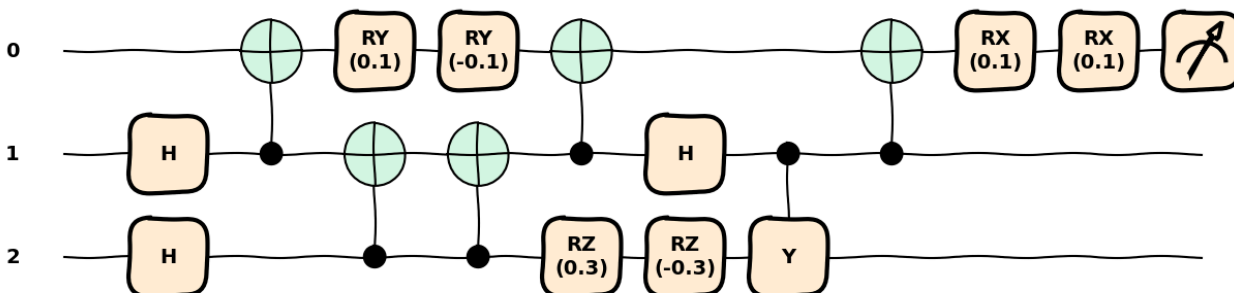
Enjoying PennyLane? Take the [2026 Unitary Foundation Quantum Open Source Software Survey](#) now.



Given an arbitrary quantum circuit, it is usually hard to clearly understand what is really happening. To obtain a better picture, we can shuffle the commuting operations to better distinguish between groups of single-qubit and two-qubit gates. We can easily do so by applying the `commute_controlled()` transform, which pushes all single-qubit gates towards a `direction` (defaults to the right).

```
new_circuit = qp.transforms.commute_controlled(circuit, direction="right")
```

```
qp.draw_mpl(new_circuit, decimals=1, style="sketch")(angles)
plt.show()
```



With this rearrangement, we can clearly identify a few operations that cancel out. For instance, the two consecutive CNOT gates on the last two wires can both be removed. This can be achieved

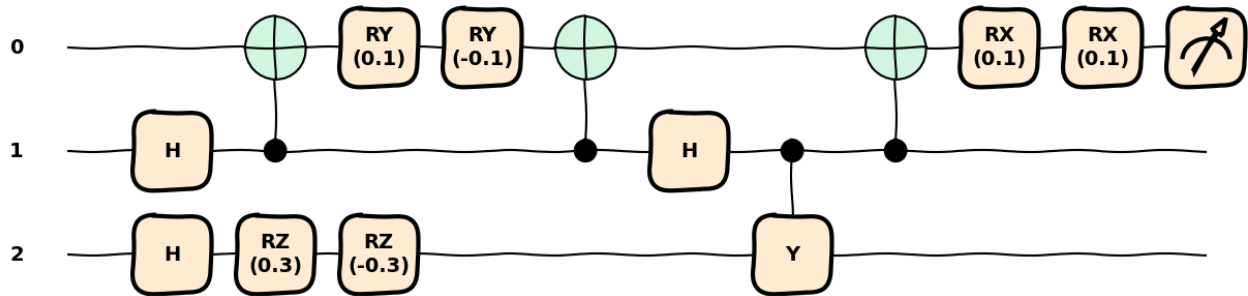


```
new_circuit = qp.transforms.cancel_inverses(new_circuit)
```

```
qp.draw_mpl(new_circuit, decimals=1, style="sketch")
plt.show()
```

Have your say!

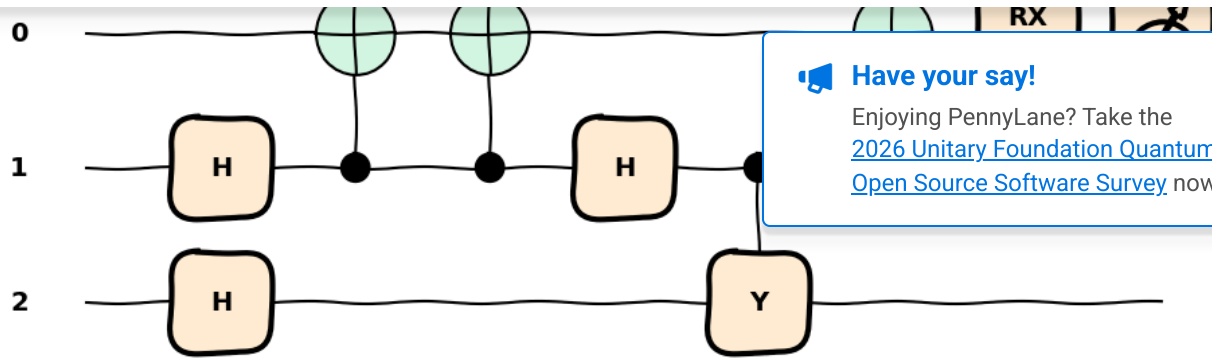
Enjoying PennyLane? Take the [2026 Unitary Foundation Quantum Open Source Software Survey](#) now.



Consecutive rotations along the same axis can also be merged. For example, we can combine the two `RX` rotations on the first qubit into a single one with the sum of the angles. Additionally, the two `RY` rotations on the first qubit and the two `RZ` rotations on the third qubit will cancel each other. We can combine these rotations with the `merge_rotations()` transform. We can choose which rotations to merge with `include_gates` (defaults to `None`, meaning all). Furthermore, the merged rotations with a resulting angle lower than `atol` are directly removed.

```
new_circuit = qp.transforms.merge_rotations(new_circuit, atol=1e-8)
```

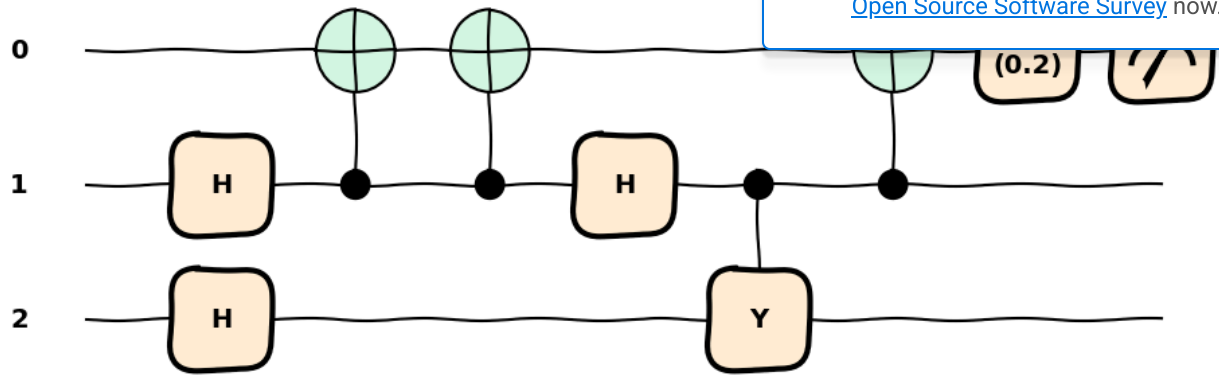
```
qp.draw_mpl(new_circuit, decimals=1, style="sketch")(angles)
plt.show()
```



Combining these simple circuit transforms, we have reduced the complexity of our original circuit. This will make it cheaper to execute and less prone to errors. However, there is still room for improvement. Let's take it a step further in the following section!

As a final remark, we can directly apply the transforms to our circuit when we define it using their decorator forms (beware the reverse order!).

```
@qp.transforms.merge_rotations(atol=1e-8)
@qp.transforms.cancel_inverses
@qp.transforms.commute_controlled(direction="right")
@qp.qnode(dev)
def qfunc(angles):
    qp.Hadamard(wires=2)
    qp.Hadamard(wires=1)
    qp.CNOT(wires=[1, 0])
    qp.CNOT(wires=[2, 1])
    qp.RZ(angles[1], wires=2)
    qp.RY(angles[0], wires=0)
    qp.CNOT(wires=[2, 1])
    qp.RY(-angles[0], wires=0)
    qp.RX(angles[0], wires=0)
    qp.RZ(-angles[1], wires=2)
    qp.CNOT(wires=[1, 0])
    qp.RX(angles[0], 0)
    qp.Hadamard(wires=1)
    qp.CY(wires=[1, 2])
    qp.CNOT(wires=[1, 0])
    return qp.expval(qp.PauliZ(wires=0))
```



 **Have your say!**

Enjoying PennyLane? Take the [2026 Unitary Foundation Quantum Open Source Software Survey](#) now.

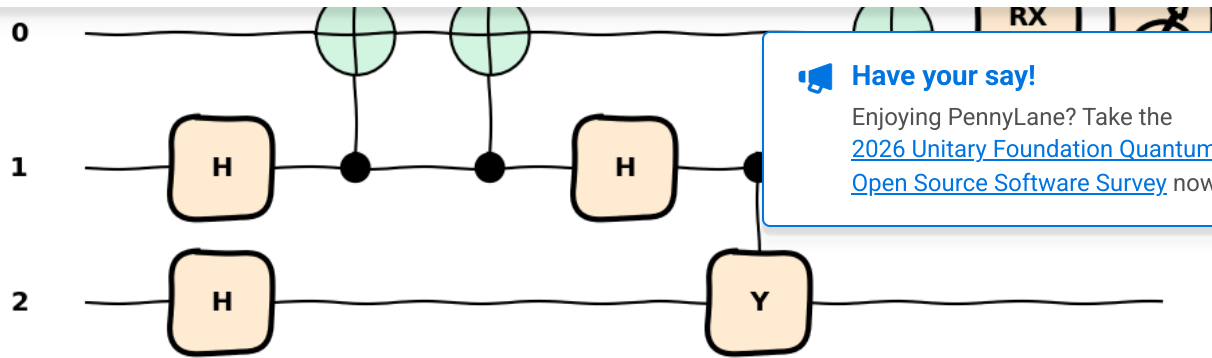
Circuit compilation ¶

A sequence of transforms is also called a compile pipeline. We can chain multiple transforms into a `CompilePipeline` that can be easily reused to transform a circuit:

```
pipeline = qp.CompilePipeline(
    qp.transforms.commute_controlled(direction="right"),
    qp.transforms.cancel_inverses,
    qp.transforms.merge_rotations(atol=1e-8)
)

compiled_circuit = pipeline(circuit)

qp.draw_mpl(compiled_circuit, decimals=1, style="sketch")(angles)
plt.show()
```



Have your say!

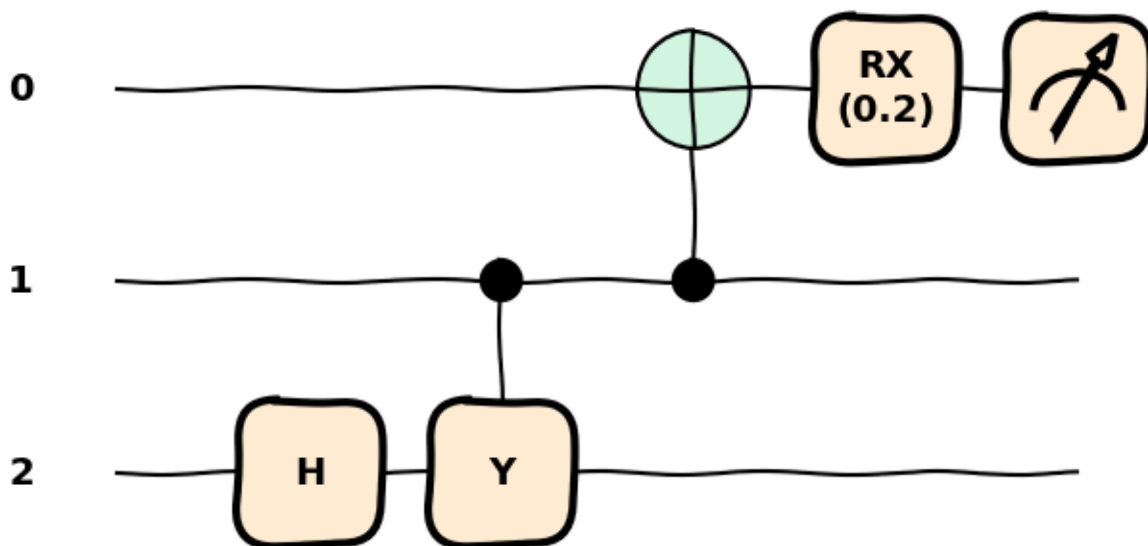
Enjoying PennyLane? Take the [2026 Unitary Foundation Quantum Open Source Software Survey](#) now.

In the resulting circuit, we can identify further operations that can be combined, such as the consecutive **CNOT** gates on the first two qubits, and then recursively the two **Hadamard** gates on the second qubit. Therefore, we could apply the pipeline a second time.

Let us see the resulting circuit with two passes.

```
compiled_circuit = pipeline(compiled_circuit)

qp.draw_mpl(compiled_circuit, decimals=1, style="sketch")(angles)
plt.show()
```





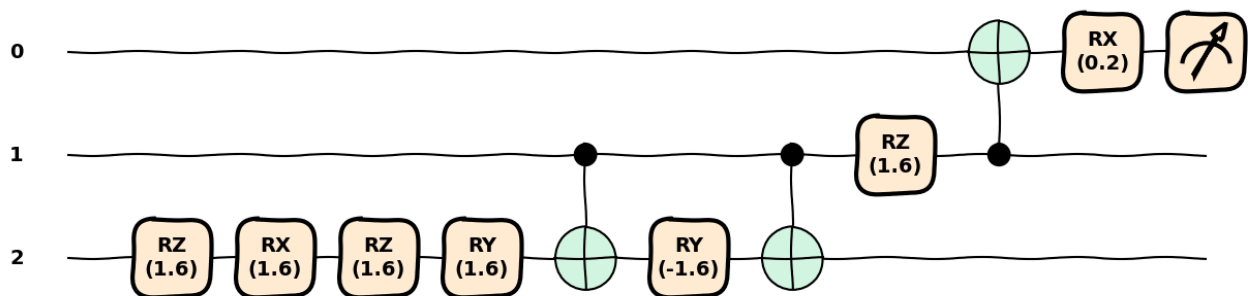
optimized circuit into a gate set so that it could be executed on a device that can only execute single-qubit rotations and the `CNOT` gate.

```
new_pipeline = pipeline * 2 + qp.transforms.decompose
compiled_circuit = new_pipeline(circuit)
```

```
qp.draw_mpl(compiled_circuit, decimals=1, style="sketch")(angles)
plt.show()
```

Have your say!

Enjoying PennyLane? Take the [2026 Unitary Foundation Quantum Open Source Software Survey](#) now.



We can see how the Hadamard and control-Y gates have been decomposed into a series of single-qubit rotations and CNOT gates. We're ready to run our circuit on the quantum device. Great job!

Conclusion ¶

In this tutorial, we have learned the basic principles of the compilation of quantum circuits. Combining simple circuit transforms that are applied repeatedly during each pass of the compiler, we can significantly reduce the complexity of our quantum circuits.

To continue learning, you can explore the documentation for the `transforms` module, which contains other circuit transformations present in the PennyLane. You can even learn how to create your own transform!

About the author ¶



Borja is a PhD student at ICFO - The Institute of Photonic Sciences, algorithms to study physics. He is also a summer resident at Xanadu.

Have your say!

Enjoying PennyLane? Take the [2026 Unitary Foundation Quantum Open Source Software Survey](#) now.

Total running time of the script: (0 minutes)

[Ask a question on the forum](#)

Share demo



0 Comments - powered by *utteranc.es*

Write

Preview

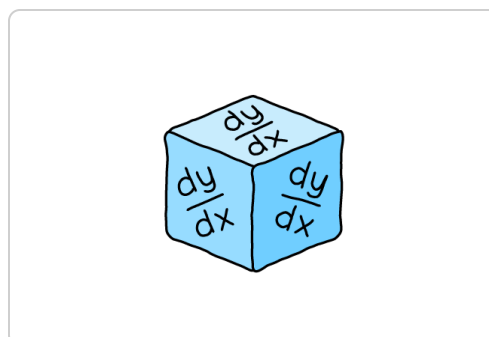
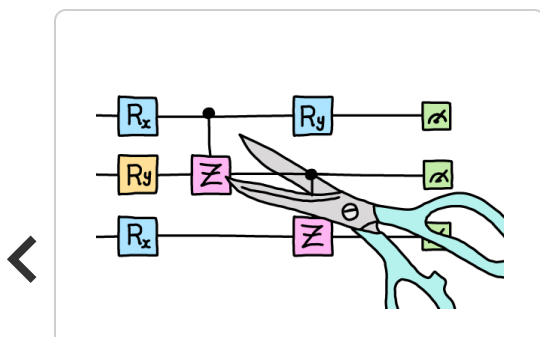
Sign in to comment



Styling with Markdown is supported

[Sign in with GitHub](#)

Related Demos



**Have your say!**

Enjoying PennyLane? Take the [2026 Unitary Foundation Quantum Open Source Software Survey](#) now.

Never miss a milestone

Get the latest quantum updates delivered to your inbox.

[Join the list](#)



PennyLane is an open-source quantum software platform for quantum computing, quantum machine learning, and quantum chemistry. Create meaningful quantum algorithms, from inspiration to implementation.

Created with ❤️ by [Xanadu](#).

Research

Research
Performance
Hardware and simulators
Demos library
Compilation hub
Quantum datasets

Education

Teach
Learn
Codebook
Coding challenges
Videos
Glossary

Software

Install
Features
PennyLane documentation
Catalyst documentation
Development guide
How-to guides
API
GitHub [↗](#)





© Copyright 2026 | Xanadu | All rights reserved

TensorFlow, the TensorFlow logo and any related marks are trademarks of Google

[Privacy policy](#)

[Terms of service](#)

[Cookies policy](#)

[Code of conduct](#)



Have your say!

Enjoying PennyLane? Take the
[2026 Unitary Foundation Quantum
Open Source Software Survey](#) now.